

Documentação Técnica de Transição

Setup, Ambiente e Infraestrutura

AcadFlow — Sistema de Gestão de Processos Acadêmicos — PPGE

Equipe 7 · Julho de 2026

0. Identificação do Projeto e da Equipe

Campo	Informação
Nome do sistema	AcadFlow — PPGEC
Repositório GitHub	https://github.com/ldhonorato/PPGEC_rec
Disciplina	Engenharia de Software
Período	2026.1
Quadro Trello	https://trello.com/b/BusBLA15/esw-sistema-de-gestao-de-processos

Equipe 7

Membro	GitHub
Bruno Morato	brunomorato
Davi de Oliveira	DaveOliviae
Ellen Beatryz	ellenbeatryzbarone
Guilherme Valença	gpvalenciaa
Isabella Nascimento	isabella-sn
Matheus Albuquerque	matheus-albuquerque-dev

1. Visão Geral da Infraestrutura e Contêineres

A aplicação é totalmente orquestrada pelo Docker Compose, com cada serviço em um ambiente isolado dentro de uma rede virtual privada.

- Gunicorn (Servidor WSGI): Recebe requisições HTTP e as processa via Django. Arquivos estáticos são servidos pelo WhiteNoise, sem Nginx na stack.
- Django (Aplicação Core): Lógica de negócios, rotas, autenticação, ORM, renderização de páginas.
- PostgreSQL (Banco de Dados): Persistência relacional. Sem Docker, o sistema usa SQLite automaticamente.
- Redis & Celery (Processamento Assíncrono): E-mails enviados ao Redis e processados em background pelo Celery Worker e Celery Beat.
- pgAdmin: Interface web para administração do banco PostgreSQL (porta 5050).

Sobre o Nginx: Em produção, o DTI gerencia um Nginx Proxy Manager externo à stack Docker que faz o proxy reverso. Os contêineres da aplicação NÃO incluem Nginx — ele é infraestrutura da universidade, não da equipe.

Arquivos de orquestração

Arquivo	Contexto	Diferença principal
<code>docker-compose.yml</code>	Desenvolvimento local	Build a partir do código local. Porta do app: 8000.

`docker-compose-prod.yml`

Produção

Puxa imagem do GHCR
(ghcr.io/ldhonorato/ppgec_rec:latest). Porta: 8001. Inclui
Watchtower para auto-update.

2. Stack Tecnológica e Ferramentas

Tecnologia / Ferramenta	Versão	Escopo	Finalidade
Python	3.11	Backend	Linguagem principal
Django	5.2	Backend	Rotas, ORM, autenticação, admin, regras de negócio
Gunicorn	latest	Servidor	Processa requisições HTTP em produção
WhiteNoise	6.4.0	Estáticos	Serve CSS/JS/imagens sem Nginx
PostgreSQL	16-alpine	Banco	Persistência em Docker e produção
SQLite	embutido	Banco	Alternativa local sem Docker
Redis	7-alpine	Broker	Fila de tarefas assíncronas para o Celery
Celery Worker	5.4.0	Assíncrono	Execução de background tasks (e-mails)
Celery Beat	5.4.0	Assíncrono	Agendador de tarefas periódicas
django-celery-results	2.5.1	Assíncrono	Persiste resultados das tasks no banco
psycopg2-binary	2.9.10	Driver	Conexão Python ↔ PostgreSQL
pgAdmin 4	8 (imagem)	Admin	Interface web de administração do PostgreSQL
Watchtower	1.7.1	DevOps	Auto-atualização de imagens Docker em produção
Docker / Compose	v2	Infra	Containerização e orquestração de serviços
python-dotenv	1.0.0	Config	Carregamento de variáveis do .env

3. Variáveis de Ambiente (.env)

⚠ Nunca versione o .env no repositório. Nunca compartilhe EMAIL_HOST_PASSWORD em PRs, issues ou canais públicos — é uma credencial real de produção.

```
#---CONFIGURAÇÕES DO NÚCLEO DA APLICAÇÃO (DJANGO)---

DEBUG=True

SECRET_KEY=[chave_secreta_local_desenvolvimento]

SITE_URL=http://127.0.0.1:8000


#---CONFIGURAÇÕES DO BANCO DE DADOS (POSTGRESQL)---

POSTGRES_DB=[nome_do_banco_de_dados]

POSTGRES_HOST=db

POSTGRES_PASSWORD=[senha_do_usuario_postgres]

POSTGRES_PORT=5432

POSTGRES_USER=[usuario_do_banco_de_dados]


#---CONFIGURAÇÕES DE ENVIO DE E-MAIL (SMTP)---

DEFAULT_FROM_EMAIL=[email_remetente_padrao]

EMAIL_HOST=smtp.gmail.com

EMAIL_HOST_PASSWORD=[senha_de_app_gmail_16_chars]

EMAIL_HOST_USER=[usuario_do_servidor_smtp]

EMAIL_PORT=587

EMAIL_USE_TLS=True


#---CONFIGURAÇÕES DO PGADMIN---

PGADMIN_DEFAULT_EMAIL=[email_valido_para_login_no_pgadmin]

PGADMIN_DEFAULT_PASSWORD=[senha_do_pgadmin]


#---CELERY (normalmente não precisam constar no .env com Docker)---

CELERY_BROKER_URL=redis://redis:6379/0
```

```
CELERY_RESULT_BACKEND=redis://redis:6379/0
```

EMAIL_HOST_PASSWORD: Use Senha de App Gmail (não a senha da conta). Gere em: Conta Google → Segurança → Verificação em duas etapas → Senhas de app. Resultado: sequência de 16 caracteres.

PGADMIN_DEFAULT_EMAIL: O pgAdmin 4.8+ valida o formato do e-mail. Use um endereço com domínio válido (ex.: admin@ppgec.com), não .local.

4. Passo a Passo: Configuração e Execução

4.1 Pré-requisitos

Instale apenas:

- Docker Desktop (Windows/macOS) ou Docker Engine + Compose v2 (Linux)
- Git

Não é necessário instalar Python, PostgreSQL ou Redis localmente.

4.2 Clonar o repositório

```
git clone https://github.com/ldhonorato/PPGEC_rec
cd PPGEC_rec
```

4.3 Criar o arquivo .env

Copie o template da seção 3 e preencha com os valores reais. Salve como .env na raiz do projeto.

4.4 Subir os contêineres

```
docker compose up --build
```

Aguarde até aparecer nos logs: "Booting worker with pid" (serviço web).

4.5 Aplicar as migrações (apenas na primeira vez)

```
docker compose exec web python manage.py migrate
```

4.6 Criar superusuário (apenas na primeira vez)

```
docker compose exec web python manage.py createsuperuser
```

4.7 Acessar o sistema

Interface	URL
Aplicação principal	http://localhost:8000
Painel Admin Django	http://localhost:8000/admin/

4.8 Comandos úteis

```
# Parar todos os serviços

docker compose down


# Parar e apagar volumes (apaga o banco de dados)

docker compose down -v


# Ver logs em tempo real

docker compose logs -f web


# Criar novas migrações após alterar models.py

docker compose exec web python manage.py makemigrations


# Acessar shell Django

docker compose exec web python manage.py shell
```

5. Criação de Usuários de Teste

Acesse o Admin Django em <http://localhost:8000/admin/> com o superusuário.

Perfil	Onde cadastrar no Admin	Observação
Aluno	Processos → Alunos	Requer TrajetoriaAcademica ativa
Docente	Processos → Docentes	Marcar coordenador=True para perfil Coordenador
Servidor (Secretaria)	Processos → Users	tipo_usuario = SERVIDOR

Fluxo mínimo para testar: 1. Criar Docente → 2. Criar Aluno → 3. Criar TrajetoriaAcademica (Aluno + Docente, status ATIVA) → 4. Configurar campo email nos Setores (Secretaria, Pleno) em Admin → 5. Logar como Aluno e abrir processo.

Não há credenciais de teste compartilhadas versionadas no projeto. Os usuários de teste devem ser criados via Admin Django conforme o fluxo mínimo acima. Credenciais de ambientes já configurados devem ser solicitadas ao responsável do projeto por canal privado (nunca compartilhadas em PR, issue ou canal público).

6. Funcionalidades Desenvolvidas pela Equipe 7

Contexto: O projeto já existia antes da Equipe 7. Esta seção cobre apenas as evoluções realizadas durante o semestre sobre a base de código preexistente.

6.1 Épico 1 — Notificações por E-mail (entregue)

Implementação completa do sistema de notificações assíncronas via Celery + Gmail SMTP. Todos os e-mails são processados em background e incluem o nome do aluno no assunto. Consulte a tabela detalhada na seção 7.7.

6.2 Infraestrutura e DevOps (entregue)

- Criação do docker-compose.yml com todos os serviços (web, db, redis, celery, celery-beat, pgadmin)
- Criação do docker-compose-prod.yml com Watchtower para auto-update de imagens
- Migração do banco de dados de SQLite para PostgreSQL 16
- Publicação da imagem Docker no GitHub Container Registry (ghcr.io)
- Disponibilização do container na porta 8001 em produção
- Apoio ao DTI na configuração do Nginx Proxy Manager externo
- Merge final e resolução de conflitos com a branch da equipe anterior

6.3 Épico 2 — Automação do Pleno (não entregue — código parcial existe)

Nenhuma feature do Épico 2 está ativa em produção, mas partes do código foram escritas e permanecem no repositório. A próxima equipe NÃO precisa partir do zero:

Feature	Status	O que existe no código
Encaminhamento ao Pleno com data limite	Revertido	Campo prazo_pleno no EncaminhamentoForm; Processo.encaminhar() aceita e valida prazo_limite.
Date picker visual no formulário	Bug ativo	Campo existe no backend mas nunca aparece na tela — erro de nome no template (Bug #1).
Notificação de intervenção no Pleno	Task existe, automação inativa	send_email_processo_comenta do_pleno está implementada e funcional. Lógica de cancelamento da automação não concluída.
Worker autônomo — aprovação silenciosa	Revertido / comentado	verificar_prazos_expirados em tasks.py está comentado. Lógica completa existe, mas foi desativada antes do go-live.

Resumo: O Épico 2 foi um trabalho extra-escopo — não estava previsto no escopo da disciplina e foi alinhado diretamente com o professor Leandro Honorato. Por não fazer parte das entregas obrigatórias do semestre, não foi concluído: o que existe é código parcial (revertido ou comentado) das features acima. A próxima equipe pode retomar a partir do código existente, mas precisa tratá-lo como ponto de partida não validado — não como funcionalidade pronta.

Estas features constituem, junto com os itens abaixo, o backlog não realizado do projeto — o ponto de partida para a próxima equipe:

- Adicionar bind=True e retry nas tasks de e-mail de setor (processos/tasks.py), que hoje não re-tentam em caso de falha de SMTP (e-mail de setor pode ser perdido).
- Corrigir o timeout do Gunicorn (--timeout 120) no docker-compose.yml, que hoje usa o padrão de 30s.
- Ajustar a rota de teste de e-mail (/teste-email/), com remetente/destinatário fixos inválidos (processos/views.py, função teste_email) — endpoint de desenvolvimento, fora dos fluxos reais.
- Reativar/definir o destinatário do e-mail de ciência efetivada para a Secretaria (task comentada em tasks.py).

7. Regras de Negócio

Nota sobre escopo: Estas regras descrevem o comportamento do sistema como um todo, incluindo módulos herdados da base preexistente e de outras equipes — não apenas o que a Equipe 7 desenvolveu. Para o que foi entregue especificamente pela Equipe 7, veja a Seção 6.

7.1 Perfis e Permissões

Ação	Aluno	Docente	Coordenador ¹	Servidor
Abrir processo	✓	✗	✗	✗
Ver processos próprios	✓	—	—	—
Ver processos de orientandos	✗	✓	✓	✓
Gerenciar caixa de entrada	✗	✗	✓ ²	✓
Encaminhar processo	✗	✗	✓ ²	✓
Solicitar ciência do orientador	✗	✗	✓ ²	✓
Deferir / Indeferir	✗	✗	✓ ²	✓
Comentar no processo (Pleno)	✗	✓ ³	✓	✓
Dar ciência (orientador)	✗	✓ ⁴	✓ ⁴	✗

¹ Coordenador = Docente com docente.coordenador = True

² Apenas quando o processo está na caixa da Coordenação ou do Pleno

³ Docentes sem cargo de coordenador só podem comentar em processos EM_DEBATE

⁴ Apenas o orientador responsável pode confirmar ou recusar a ciência

7.2 Ciclo de Vida de um Processo



Status	Descrição
EM_ANÁLISE	Estado inicial após abertura e após retorno de ciência ou documentação.
AGUARDANDO_DOCUMENTO	Secretaria/Coordenação devolveu aguardando ajuste do aluno.
AGUARDANDO_CIÊNCIA	Solicitação de ciência do orientador pendente.
EM_DEBATE	Processo encaminhado ao Pleno — docentes podem comentar.
FINALIZADO	Deferido ou indeferido com termo de finalização.

Regras importantes:

- Não é possível encaminhar um processo finalizado.
- Não é possível encaminhar com ciência do orientador pendente.
- Ao deferir ou indeferir, um termo de finalização é obrigatório.

7.3 Numeração de Processos

Número gerado automaticamente no formato YYYYMM-NNNNNN (ex.: 202507-000001). A sequência reinicia a cada mês. A geração é protegida por `select_for_update` para evitar duplicatas em concorrência.

7.4 Solicitação de Ciência do Orientador

- Só pode ser solicitada por Servidor ou Coordenador.
- Só pode existir uma solicitação pendente por vez por processo.
- O processo muda para AGUARDANDO_CIÊNCIA automaticamente.
- Ao orientador confirmar ou recusar, o processo volta para EM_ANALISE.
- O orientador é determinado pela `TrajetoriaAcademica` ativa do aluno criador.

7.5 Documentos e Restrição de Acesso

Documentos podem ser marcados com 7 categorias de restrição (Lei 12.527/2011):

Quem pode visualizar um documento restrito

Quem enviou o documento

Servidor (Secretaria) — acesso irrestrito

Coordenador (`docente.coordenador = True`)

Alunos e Docentes comuns NÃO veem documentos restritos de outros. A remoção de arquivo exige motivo obrigatório e é rastreada (quem removeu, quando, por quê).

7.6 Trajetória Acadêmica e Orientador

O vínculo entre Aluno e Orientador NÃO fica no modelo Aluno — fica em `TrajetoriaAcademica`. Isso tem impacto direto em queries:

```
#  Correto

Aluno.objects.filter(

    trajetorias__orientador=request.user,

    trajetorias__status=TrajetoriaAcademica.Status.ATIVA,

).distinct()

#  Errado — campo não existe mais

Aluno.objects.filter(orientador=request.user)
```

7.7 Notificações por E-mail

E-mails disparados de forma assíncrona via Celery. O nome do aluno é incluído em todos os assuntos.

Remetente institucional: Todos os e-mails do sistema são enviados a partir do endereço institucional acadflow@ecomp.poli.br, configurado na variável EMAIL_HOST_USER do arquivo .env. Esse é o endereço que aparece no campo "De:" das notificações.

Resumo por evento — quem recebe e-mail em cada ação:

⚠ Uma única ação no sistema pode disparar vários e-mails simultaneamente para destinatários diferentes. A tabela abaixo mostra o fan-out completo de cada evento (validado no código-fonte).

Ação no sistema	Quem recebe e-mail	Tasks disparadas	Local no código
Abertura de processo	Aluno criador · Orientador · Secretaria (setor inicial)	novo_processo_aluno + novo_processo_orienta dor + novo_processo_secretaria	views.py:1909-1911 / 2513-2515 / 2576-2578
Encaminhamento para setor comum	Aluno criador · Orientador · Setor de destino	movimentacao_aluno + movimentacao_orienta dor + mudanca_setor	views.py:1680-1685
Encaminhamento ao Pleno	Aluno criador · Orientador · Setor de destino · Todos os docentes	movimentacao_aluno + movimentacao_orienta dor + mudanca_setor + movimentacao_pleno	views.py:1680-1685
Devolução ao requerente	Aluno criador · Orientador	devolucao_requerente + movimentacao_orienta dor	views.py:1677-1678 + 1685
Solicitação de ciência	Orientador responsável	solicitacao_ciencia	views.py:1595
Manifestação de ciência (muda status, não setor)	Setor atual (via setor.email)	status_atualizado	views.py:1622-1627
Comentário/intervençã o no Pleno	Todos os docentes	processo_comentado_p leno	views.py:1756
Finalização (deferido/indeferido)	Aluno criador · Orientador	conclusao_aluno + conclusao_orientador	views.py:1534-1550 / 1711-1712
Nova solicitação de assinatura	Docente destinatário ou e-mail do setor + membros ativos	solicitacao_assinatura	views.py:2397

Detalhe do encaminhamento: O e-mail para o orientador (movimentacao_orientador) é disparado sempre, inclusive na devolução ao requerente (está fora do if/else em views.py:1685). Já o e-mail ao setor de destino (mudanca_setor) NÃO é enviado na devolução ao requerente — apenas em encaminhamentos normais.

As tabelas abaixo detalham cada task individualmente.

E-mails para pessoas (endereço pessoal):

Evento	Destinatário	Task
Processo aberto	Aluno (criador)	send_email_novo_processo_aluno
Processo aberto	Orientador (via TrajetoriaAcademica)	send_email_novo_processo_orientador
Solicitação de ciência	Orientador responsável	send_email_solicitacao_ciencia
Devolução para ajustes	Aluno (criador)	send_email_devolucao_requerente
Movimentação	Aluno (criador)	send_email_movimentacao_aluno
Movimentação	Orientador (via TrajetoriaAcademica)	send_email_movimentacao_orientador
Processo finalizado	Aluno (criador)	send_email_conclusao_aluno
Processo finalizado	Orientador (via TrajetoriaAcademica)	send_email_conclusao_orientador
Novo processo no Pleno	Todos os Docentes cadastrados	send_email_movimentacao_pleno
Intervenção no Pleno	Todos os Docentes cadastrados	send_email_processo_comentado_pleno

E-mails para setores (e-mail institucional via Setor.email):

Evento	Destinatário	Task
Processo aberto	Setor inicial (Secretaria)	send_email_novo_processo_secretaria
Processo tramitado	Setor de destino	send_email_mudanca_setor
Status atualizado	Setor atual	send_email_status_atualizado

Solicitações de assinatura:

Evento	Destinatário	Task
Nova solicitação de assinatura	Docente (se destinatário docente) ou e-mail do setor + membros ativos	send_email_solicitacao_assinatura

Desabilitado (comentado em tasks.py):

Evento	Destinatário original	Motivo
Ciência efetivada pelo orientador	Secretaria	Comentado por decisão da equipe — endereço de destinatário não finalizado

Como o e-mail institucional dos setores funciona: O campo Setor.email (configurável em Admin → Processos → Setores) é o endereço para o qual as notificações do setor são enviadas. As tasks verificam "if setor.email:" antes de disparar — se vazio, o e-mail é silenciosamente ignorado. É OBRIGATÓRIO configurar esse campo no Admin para que Secretaria e setores recebam notificações.